

COMP232 Lab 3

Running time of JCA library functions

For this lab session, you will learn to estimate the running time of password-based encryption/decryption functions in JCA. To do this, you will first need to learn about measuring running time in Java. There are several ways of doing this. You should experiment with them to get comfortable with these constructs.

Measuring Execution time in Java

You can use either `System.nanoTime()` to measure elapsed time or `System.currentTimeMillis()` to measure wall-clock time. A good discussion of differences between these two, and several other methods, with examples of code can be found at the link in the References section below. Observe that for most of these methods, you will need to compute the difference between end-time and start-time similar to how you might use a wall clock to measure elapsed time.

Measuring Execution time in Python

You can use `time.perf_counter()` or `timeit.timeit()` to measure execution time of small code snippets in Python .

References:

- [JCA/JCE Reference Manual](#)
- [Comparison of different functions for measuring execution times](#)
- [Python Reference Manual](#)

Task 1

Modify the Password-Based Encryption Program `PBEs.java` or `PEB.py` used in Lab 2, so it can measure execution time of

- Key generation and encryption (repeated 5 times);
- Key generation and decryption (repeated 5 times).

Measure the above times for a fixed password, fixed plain text, salt and iteration count. Repeat experiments a few times for each measurement. Make a record of your measurements.

Task 2

- Change your program to try different iteration counts: 1, 2, 8, 64, 256, 65536, etc. How does it affect time measurements above?
- Try various passwords. Are time measurements affected?
- Try plain texts of different sizes. Are time measurements affected?

Question:

Should we increase or decrease iteration count to make an encryption more resistant to brute-force search attack? What are potential disadvantages of doing that?

Task 3 (After the lab): Take user input.

- Learn about the `System.console().readPassword()` method in Java or the `getpass.getpass` method in Python to take passwords as input from the user.
- Take input from user for both encrypting and decrypting.
- Output the decrypted text, and observe what happens when the password entered for decryption does not match the one entered for encryption.
- Learn how to write a simple try-catch block in Java or try-except block in Python to handle exceptions. Implement this to handle mismatched passwords.
- Extend this further to take the plaintext to be encrypted also as input from console.